

Flujo completo de Login + JWT — del frontend a cada microservicio

Documento-guía para **recorrer el código** del inicio de sesión de punta a punta: cómo el frontend pide el token, cómo `users-service` lo **emite** (firma), cómo vuelve y se guarda, y cómo **cada servicio lo valida** en los requests siguientes. Cada paso indica el **archivo y las líneas** exactas para abrir y leer.

El flujo de un vistazo

```
[1-3] Frontend: el usuario envía email/password → POST /api/auth/login
      |
[4]   api-gateway rutea /api/auth/** → users-service
      |
[5-8] users: AuthController → AuthService (verifica password BCrypt)
      → JwtService.generateToken (FIRMA con el secreto) → AuthResponse {token}
      | (el token vuelve al front)
[9-10] Frontend: guarda el token en localStorage (y decodifica el rol para la UI)
      |
[11]  En CADA request siguiente, un interceptor agrega "Authorization: Bearer <token>"
      |
[12-14] Cada servicio (catalog, order, payment, review, promo) VALIDA el token:
      filtro lee el header → verifica la firma con el MISMO secreto → arma la identidad
      |
[15]  Entre servicios (Feign), el token se REENVÍA para seguir autenticado aguas adentro
```

Dos mitades: **emisión** (pasos 1–8, una sola vez al loguearse) y **validación** (pasos 11–15, en cada request). El "pegamento" es el **secreto JWT compartido**: `users` firma con él, todos verifican con él.

FASE 1 — Frontend: el usuario se loguea

#	Archivo : línea	Qué pasa / qué buscar
1	frontend/src/features/auth/LoginPage.tsx:5	Renderiza <code><AuthForm mode="login" /></code> . La lógica del formulario (inputs email/password) está en <code>AuthForm.tsx</code> ; al enviar llama <code>useAuth().login(...)</code> .
2	frontend/src/auth/AuthContext.tsx:75-77	La función <code>login()</code> : <code>await api.post('/api/auth/login', credentials)</code> . Acá arranca la llamada al backend.
3	frontend/src/config.ts:5-6	<code>API_BASE_URL = http://localhost:8080</code> (el gateway). Es a donde pega el front.

FASE 2 — Backend: el gateway rutea y `users` emite el token

#	Archivo : línea	Qué pasa
---	-----------------	----------

4	services/api-gateway/src/main/resources/application.yml:24-27	Ruta users-auth: Path=/api/auth/** → lb://users-service. El gateway resuelve users-service por Eureka y le manda el login.
5	services/users-service/.../controller/store/AuthController.java:32-34	@PostMapping("/login") → delega todo a authService.login(request). El controller no tiene lógica.
6	services/users-service/.../service/impl/AuthServiceImpl.java:60-71	El corazón del login: busca por email (63), verifica el password con BCrypt (passwordEncoder.matches, 65), chequea que el usuario esté activo (68), y en línea 71 llama jwtService.generateToken(user) y arma el AuthResponse. <i>(Mismo mensaje de error para email inexistente / pass mal / inactivo: no se filtra qué emails existen.)</i>
7	services/users-service/.../security/JwtService.java:32-40	generateToken() : subject(email) + claim("role", ...) (el rol viaja DENTRO del token) + issuedAt/expiration + signWith(key) (línea 39 = FIRMA HMAC) + compact() (40 = arma header.payload.signature). La clave se deriva del secreto en línea 28 (Keys.hmacShaKeyFor(secret)).
8	services/users-service/.../dto/response/AuthResponse.java	El DTO que vuelve al front: { token, type: "Bearer", expiresIn }.

FASE 3 — Frontend: recibe y guarda el token

#	Archivo : línea	Qué pasa
9	frontend/src/auth/AuthContext.tsx:68-71	Con la respuesta: decodifica (userFromToken(res.token)) y guarda el token: localStorage.setItem(TOKEN_KEY, res.token) (71).
10	frontend/src/auth/AuthContext.tsx:20-29	decodeToken: parte el header.payload.signature y decodifica el payload (base64url) solo para LEER el rol y mostrar/ocultar cosas en la UI. El front NO verifica la firma — confiar en el payload sin verificar es solo para la interfaz; la seguridad real la pone el backend.

FASE 4 — Frontend: envía el token en cada request

#	Archivo : línea	Qué pasa
11	frontend/src/lib/api.ts:13-16	El interceptor de axios : antes de CADA request, lee el token de localStorage y agrega el header Authorization: Bearer <token> . Esto es lo que hace que el token viaje automáticamente en todas las llamadas.

FASE 5 — Backend: cada servicio VALIDA el token (ejemplo: catalog)

#	Archivo : línea	Qué pasa
12	services/catalog-service/.../security/JwtAuthenticationFilter.java:13-21	El filtro (corre una vez por request, ANTES de la a lee Authorization (14), saca el token tras "Bearer " jwtService.isValid(token) (17), arma la autoridad (18) y mete la identidad en el SecurityContextHolder token no es válido, no autentica → la capa de aut responde 401.
13	services/catalog-service/.../security/JwtService.java:11-14	isValid() (11) → parse() (14) hace Jwts.parser().verifyWith(key).build().parseSigned recalcula la firma con el secreto compartido y la la del token, y chequea la expiración. Si algo falla, excepción → isValid devuelve false. extractRole (claim role.
14	services/catalog-service/.../config/SecurityConfig.java:51-61	STATELESS (51, sin sesión), las reglas de qué endpo auth/rol (53-57: GET de catálogo público, /api/adm ADMIN, el resto autenticado), y addFilterBefore(j ...) (61) que engancha el filtro antes del de Spir

Atajo: order , payment , review y promo tienen su JwtService , JwtAuthenticationFilter y SecurityConfig **idénticos** a los de catalog (la Fase 5 es la misma en todos). Solo cambian las reglas de rutas en cada SecurityConfig . Entendido catalog, los demás son calcados.

BONUS — el token viaja servicio→servicio (Feign)

#	Archivo : línea	Qué pasa
15	services/order-service/.../config/FeignAuthInterceptor.java:20-24	Cuando order llama a otro servicio por Feign (catalog para el stock, payment para el cobro, promo para el cupón), este interceptor reenvía el mismo Authorization (22-24). Sin esto, las llamadas internas saldrían sin token y el otro servicio respondería 401. Así el token sigue viajando aguas adentro.

Qué se valida exactamente (recordatorio)

Cuando un servicio recibe Authorization: Bearer <token> , "válido" significa **dos cosas juntas**:

1. **La firma coincide.** El servicio recalcula HMAC-SHA384(base64url(header) + "." + base64url(payload), secreto) y compara con la 3ª parte del token. Como usa el **mismo secreto** que users , le da igual → confía. Si el payload fue alterado (o el secreto difiere), la firma no coincide → **401**.
2. **No está expirado.** El claim exp debe estar en el futuro.

El payload (email + rol) viaja **firmado pero no encriptado**: cualquiera lo lee, pero **nadie lo puede modificar** sin el secreto. Por eso catalog confía en el role del token sin preguntarle a users ni tocar la base de datos. El

detalle de la firma HMAC está en la sección "JWT a fondo" de [guia-implementacion-microservicios.md](#) .

Resumen del recorrido

